Inverting Python: Designing NTP-Prior-Resistant Programming Languages toward LLM-Resistant Coding Assessment

Jaeyeong CHOI
DGIST EECS
jaeyeong2022@dgist.ac.kr

***Abstract*—LLMs now solve standard programming tasks well enough to undermine their value as student assessments. We investigate whether a human-interpretable language can defeat LLM next-token-prediction (NTP) priors. We propose a four-level taxonomy—L1 (token alias), L2 (syntax inversion), L3 (explicit semantic inversion), L4 (implicit semantic inversion)—and evaluate eleven models (seven OpenAI core, two frontier, two open-weight). L4 defeats *all* tested models on prior-entrenched tasks through *pattern blindness*—including `gpt-5.4` and `o3` (0/10 each on Fibonacci). A multi-task extension (345 runs) reveals task-dependent resistance and a third failure mode, *operational substitution*. An annotation density experiment (E34, 150 runs; preliminary, n=10 per condition) finds that code-example mimicry outperforms verbal rule annotation, though non-monotonic patterns may partly reflect sampling variance. I/O opacity (E35, 60 runs) provides a complementary resistance pathway: opaque I/O pairs alone yield complete resistance (0/30). A chain-of-thought ablation (160 runs) reveals *reasoning–generation dissociation*: models articulate the inversion rule yet still generate standard Python. Across 1,140+ L4 evaluations we identify three failure mechanisms: *prior dominance*, *pattern blindness*, and *operational substitution*. All evaluations and artifacts are publicly available at https://github.com/Jaeyeong-CHOI/cse307-open-project.**

***Index Terms*—confusion language, LLM resistance, NTP prior, semantic inversion, programming education, code assessment**

I. Introduction

Modern code-generation benchmarks [1], [2] show that large language models (LLMs) can solve standard programming tasks with high accuracy. This creates a critical challenge for programming education: if LLMs solve assignments faster and more accurately than students, how can instructors meaningfully assess student understanding?

This paper studies a complementary but understudied question:

> *Is it possible to design a programming language that is human-interpretable but systematically defeats LLM next-token-prediction priors—making it resistant to AI-based code generation?*

The core asymmetry. The key insight is that humans and LLMs fail in fundamentally different ways when given an unfamiliar language rule. A human student who reads the rule *"if-blocks execute when the condition is **false**"* can immediately apply it—the rule is explicit, verifiable, and overrides prior habit. An LLM given the identical rule in its prompt instead produces standard Python: its next-token prediction (NTP) prior, built from billions of Python training tokens, systematically overrides the stated instruction. This asymmetry—*humans follow novel rules; LLMs revert to training priors*—is the foundation of our resistance framework and requires no additional experiment to accept: it follows directly from the mechanics of NTP-based generation.

We decompose this into three sub-questions:

1) Does changing token identifiers (L1: alias substitution) create effective LLM resistance?
2) Does inverting syntactic structure (L2) create effective resistance?
3) Does inverting semantic behavior (L3/L4) create deeper resistance?

Our contributions are:

1) **L4 Pattern Blindness as a universal LLM failure mode.** L4 (implicit semantic inversion) defeats *all* eleven tested models—including frontier models `gpt-5.4` and `o3`—on prior-entrenched tasks (0/350 pass on Fibonacci under strict example-only delivery). Models cannot extract inverted semantics from worked examples, even when chain-of-thought reasoning correctly articulates the rule.
2) **An L1–L4 resistance taxonomy with three failure mechanisms.** We identify *prior dominance* (prior overrides rules), *pattern blindness* (model fails to extract semantic rules from examples), and *operational substitution* (correct output via arithmetic workaround). Token-level (L1) and syntactic (L2) perturbations are insufficient; semantic inversion (L3/L4) is required for robust resistance, modulated by prior entrenchment depth and I/O transparency.
3) **Two complementary resistance pathways.** Beyond L4 prior exploitation, I/O opacity (E35) provides a second, independent resistance source: opaque I/O pairs alone produce complete resistance (0/30, 0%) regardless of prior strength.
4) **Empirical characterization across 1,140+ evaluations.** Eleven models (9 OpenAI + 2 open-weight), 35 experiments, three tasks, and ablations on annotation density, cross-task transfer, and chain-of-thought—providing the first systematic map of where and why NTP-prior-based resistance succeeds or fails.

II. Related Work

A. Code Generation Benchmarks and Evaluation

HumanEval [1] and MBPP [2] established functional correctness as the standard metric for code-generation evaluation. More recent benchmarks scale and diversify tasks: BigCodeBench [3] adds complex, multi-function instructions, and LiveCodeBench [4] avoids contamination through continuous updating. Across all these benchmarks, Python dominates training corpora, embedding strong token-level priors into all models evaluated. This motivates our L1 baseline: even with explicit alias instructions, models leak Python keywords at 26–69%.

B. Code-Oriented LLM Architectures and Training

Pre-trained code models—CodeBERT [5], CodeT5 [6], and InCoder [7]—demonstrate that NTP objectives on Python code reinforce keyword position priors. More recent instruction-tuned models (WizardCoder [8], DeepSeek-Coder [9]) amplify these priors through even larger code corpora. This trajectory predicts our finding: *stronger code LLMs do not necessarily mean better alias compliance*—gpt-5.4-mini (KLR=0.67) underperforms gpt-4o (KLR=0.37).

C. Instruction-Following in Code (2024–2025)

CodeIF [10] and MultiCodeIF [11] introduce benchmarks measuring LLM adherence to developer-specified constraints in code generation tasks. Critically, MultiCodeIF reports that models perform relatively well on *explicit, concrete* constraints but degrade substantially on *implicit or abstract* constraints, and that performance drops sharply as the number of hierarchical constraints increases. This explicit–implicit asymmetry is the closest mechanistic evidence in the literature to our L3/L4 findings: models can follow explicit semantic inversion rules (L3, with some model-capacity dependence), yet fail entirely when the same semantic inversion is delivered only through examples without explicit articulation (L4, pattern blindness across all tested models). The key distinction is scope: CodeIF and MultiCodeIF benchmark *constraint-following* within standard programming tasks, whereas our work *designs* a programming language whose semantic structure systematically conflicts with NTP priors, making resistance a structural property rather than a task-level benchmark artifact.

D. Instruction Hierarchy and Prior Conflicts

Chen et al. [12] propose an instruction hierarchy to resolve conflicts between system-level and user-level instructions. Their framework addresses natural-language directive conflicts; ours involves programming-language semantic rule conflicts—a dimension not studied in alignment work. Xie et al. [13] and Wu et al. [14] show that when context conflicts with parametric knowledge, LLMs often default to pretraining priors. Zhang et al. [15] survey this knowledge conflict problem broadly. Shi et al. [16] demonstrate LLMs are easily distracted by irrelevant context; our L4 condition represents the inverse: models *fail to use* relevant semantic cues provided in examples.

E. NTP Priors and In-Context Learning

Wang et al. [17] formalize a "law of next-token prediction" providing theoretical grounding for how NTP training shapes output distributions. Min et al. [18] show ICL demonstrations affect outputs primarily through distributional surface patterns rather than semantic content—explaining why models fail to extract inverted semantics from L4 examples. Dong et al. [19] survey ICL comprehensively. Lampinen et al. [20] document "A-Not-B" errors where LLMs fail to suppress previously established response patterns under context changes, directly analogous to our L4 pattern blindness.

F. Counterfactual and Semantic Reasoning Failures

Zhao et al. [21] show LLMs struggle when counterfactual reasoning requires overriding parametric knowledge—the same mechanism we observe at L3 for weak models. Turpin et al. [22] show LLM self-explanations are often unfaithful to actual behavior. Valmeekam et al. [23] document reasoning rigidity—models revert to memorized strategies under redefined rules—providing further precedent for our pattern blindness finding.

G. Adversarial Perturbation and Code Resistance

PromptBench [24] demonstrates LLM fragility under text perturbations. Pan et al. [25] show semantically aligned code prompt perturbations cause up to 42% accuracy drops. Jha et al. [26] use structured adversarial prompts to break LLM reasoning in code tasks. Khan et al. [27] specifically design prompt perturbations to reduce LLM correctness on programming assignments by 77%, most directly related to our goal. Our approach is complementary: instead of perturbing prompts, we design a confusion language whose NTP prior conflict is a *structural property*, making it principled and reproducible.

H. DSL and Constrained Code Generation

Wu et al. [28] survey LLM performance on low-resource and domain-specific languages, confirming that Python-trained models underperform on less common languages. ConCodeEval [29] benchmarks code constraint comprehension in DSLs; L2CEval [30] evaluates language-to-code generation across specialized languages. Constrained decoding [31], [32] enforces grammar-level output validity. Our L2 experiment shows models adopt explicit syntax rules entirely, but semantic constraints (L3/L4) prove far more resistant—showing grammar-level constraints are insufficient without semantic-level challenges.

I. Real-World Engineering Benchmarks

SWE-bench [33] reveals that LLMs fail to generalize from canonical benchmarks to real repository-level tasks—confirming that high performance on standard benchmarks does not indicate robustness to structural novelty, as we observe in our L1 L4 evaluations.

J. LLM in Programming Education

Dakhel et al. [34] show LLM coding tools are an "asset or liability" depending on student expertise. Lau and Guo [35] document novice programmers adopting LLM outputs uncritically. Prather et al. [36] show mixed student perceptions. Liffiton et al. [37] propose guardrailed LLM tutoring. These studies confirm a shared concern: LLM integration can undermine independent skill development.

K. Academic Integrity and LLM-Resistant Assessment

Novak and McDanel [38] propose contextual, personalized assignments to resist LLM cheating—the most directly related paper to our educational goal—but without a formal theory of what features drive LLM failure. Code perplexity detection [39] is brittle; our taxonomy offers a principled design-based solution. RLHF-based alignment [40] improves instruction-following but, as our L3/L4 results show, remains insufficient when semantic inversions must be extracted from examples.

L. Specification Gaming and Reward Hacking

Krakovna et al. [41] catalogue specification gaming behaviors where agents exploit loopholes rather than following intended objectives. Our "operational substitution" failure mode—where models produce correct output via arithmetic workaround instead of adopting the semantic rule—is a code-generation analogue of specification gaming: the model satisfies the output specification without acquiring the intended semantic competence.

M. Adversarial Code, Prompt Injection, and Jailbreaking

Prior adversarial code work (PromptBench [24], Pan et al. [25]) focuses on *syntactic* perturbations—renaming variables, reordering tokens—that degrade generation quality. Our approach targets *semantic* inversion: the surface syntax remains standard Python, but the conditional semantics are reversed, representing a fundamentally different threat model. L3/L4 also share structural similarity with prompt injection [12]: both present in-context instructions that conflict with the model's default behavior. However, the goal is inverted—in jailbreaking, the attacker wants the model to *override* its safety prior; in our framework, we want the NTP prior to *win*, producing standard Python despite the inverted-semantics instruction. L4 pattern blindness thus provides evidence that prior override is *asymmetric*: safety alignment can be bypassed by sufficiently crafted prompts, but code-generation priors resist override even when the user *wants* compliance.

N. Research Gap

Prior adversarial work degrades generation quality without principled language design. Prior DSL work measures performance without explaining failure mechanisms. Prior knowledge-conflict work focuses on factual knowledge, not programming-language semantics. Our contribution closes these gaps: a theory-grounded L1–L4 taxonomy that characterizes LLM failure by mechanism (prior dominance vs. pattern blindness) and guides principled LLM-resistant language design.

TABLE I: Four-level confusion language taxonomy. Each level targets a distinct linguistic dimension and delivery modality. L4 is the only level where all tested models fail regardless of capacity.

Level	Perturbation	Delivery	Observed failure mode
L1	Token alias	Explicit	Prior dominance (partial)
L2	Syntax inv.	Explicit	None—adopted
L3	Semantic inv.	Explicit	Capacity-dependent
L4	Semantic inv.	Implicit	Pattern blindness

Note: L4 experiments use three distinct sub-conditions across sections: (1) strict example-only ablation (§V-D, $n = 50$), the canonical L4 condition; (2) partially-annotated multi-task (§V-E), a weakened form with inline comment hints; and (3) hard-task extension (§V-M), example-only on algorithmically complex tasks. Findings differ by sub-condition; see §VI-C for synthesis.

III. Method

A. Research Questions and Hypotheses

RQ1: At what language design level does LLM resistance become robust? **RQ2**: What failure mechanism is active at each level—prior dominance or pattern blindness?

H1: Token-level alias substitution (L1) creates partial but not complete resistance. **H2**: Syntactic inversion (L2) is insufficient—models adopt new syntax when given explicit rules. **H3**: Semantic inversion (L3/L4) provides robust resistance, but differs in mechanism by model capacity.

B. Language Level Taxonomy

We design a structured family of programming language variants with properties engineered to conflict with LLM next-token prediction priors—which we term a *confusion language* (our proposed terminology).[1] We distinguish confusion languages from esoteric languages (e.g., Brainfuck), which aim for human uninterpretability; confusion languages retain full human interpretability while selectively targeting LLM NTP priors. Each confusion language variant defines rules along two axes: (1) *perturbation depth* (token, syntax, semantic) and (2) *rule delivery* (explicit vs. implicit). Table I summarizes the resulting four-level taxonomy.

Figure 1 illustrates the two-axis design space and the location of each level. The two gray cells (Token/Implicit and Syntax/Implicit) are theoretically possible configurations that we did not study; Token/Implicit would expect higher KLR than L1-Explicit (weaker contextual cues), while Syntax/Implicit would expect higher SIR. We leave these as open positions in the design space for future work.

C. L1: Token Alias Substitution

Python keywords (`if`, `for`, `in`, `return`, `def`) are replaced with aliases. Constraints: one-to-one mapping, no alias collision, token-boundary replacement, literal/comment protection.

[1]A confusion language does not require a new grammar or syntax. L1–L3 reuse Python surface syntax entirely, with behavioral differences arising from redefined semantic mappings delivered via prompt context. The only syntactic deviation tested is the L2 extension (`:define f [args] ->` block delimiters), which is given to the model as explicit grammar rules. L4 uses standard Python syntax with no syntactic modifications.

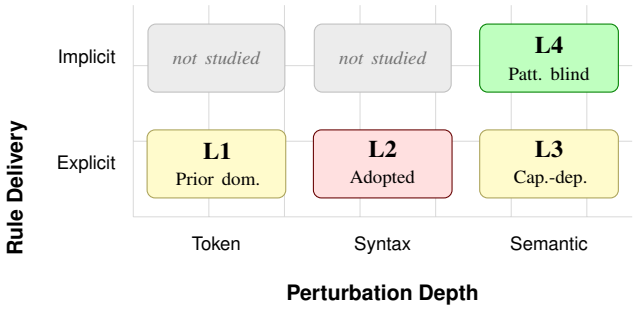

Fig. 1: Two-axis confusion language design space. Perturbation depth (token/syntax/semantic) and rule delivery (explicit/implicit) together determine LLM failure mode. L4 (green = most resistant) occupies the hardest cell: semantic perturbation without explicit rule—the only condition where all tested models fail.

Python (standard)

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

Standard semantics: if-block runs when condition is TRUE

L3 (semantic inversion)

```
def fib(n):
    if n <= 1:
        return fib(n-1) + fib(n-2)
    return n
```

Inverted: if-block runs when condition is FALSE

Fig. 2: Semantic inversion in L3. Surface syntax is identical to Python, but if-block semantics are inverted. The correct L3 solution (right) looks counter-intuitive to Python-trained models, which generate the left version regardless of the stated rule.

D. L2: Syntax Structure Inversion

Function definitions, conditionals, loops, and block delimiters are structurally inverted relative to Python. Example: `def fib(n):` becomes `:define fib [n] ->`. Model is given explicit grammar rules.

E. L3: Semantic Inversion (Explicit Rules)

Python surface syntax is retained but the semantics of if-blocks are inverted: the indented block executes when the condition is *false*. Model is explicitly told this rule. Figure 2 illustrates the concept.

F. L4: Semantic Inversion (Implicit, Example-Only)

Same as L3, but no rule is stated. Instead, models receive 2–3 worked examples that demonstrate the inversion. Models must extract the semantic pattern through few-shot reasoning.

G. Evaluation Metrics

Redundant metrics are removed. We report two independent measures:

- **KLR** (Keyword Leakage Rate): fraction of alias slots where original Python keyword was used. Note: ACR $= 1 -$ KLR and PIR $\approx$ KLR are derived quantities and not separately reported.
- **PSS** (Partial Structural Score, exploratory): weighted sub-component credit (alias: 0.4, parse: 0.25, skeleton: 0.25, format: 0.1). Weights are heuristic; PSS is reported as a secondary diagnostic alongside KLR and should not be used as the sole basis for compliance conclusions.
- **PPR** (Python Prior Rate, L3/L4): fraction of samples using Python-style condition (no inversion).
- **PES** (Prior Entrenchment Score, exploratory): probability assigned by the model to the canonical Python token at the key decision point of a task's base case, measured via the logprobs API (`top_logprobs=5`, `max_tokens=1`, `temperature=0`). PES quantifies the strength of the prior that L4 must overcome; see E31 results in §V-I.

H. Threats to Validity

Internal: judge rules may be stricter than human acceptance; PSS weights are heuristic. **Construct**: multi-task extension (T1 fib, T2 is_sorted, T3 count_vowels) partially addresses single-task limitation; broader coverage (nested control-flow, stateful logic) remains for future work. Open-weight evaluation (`Llama-3.3-70B`, `Qwen3-32B`) is reported with extended 5-variant runs (§V-H): `Qwen3-32B` completed 50/50 valid runs (full replication); `Llama-3.3-70B` achieved 25/50 valid runs ($n_{\text{eff}} = 5$ per variant due to Groq rate limiting), insufficient for per-variant statistical claims. **External**: primary results cover nine model families (seven OpenAI + two open-weight via Groq); `Llama-3.3-70B` per-variant results remain underpowered and full replication is deferred to future work. Human baseline (Exp-5) needed to validate the educational claim.

IV. Experiments

A. Experimental Overview

We ran experiments at each language level for available model families. All runs used prompts from `v1-v120` (L1) or dedicated level-specific banks (L2–L4). API authentication was managed via project environment variables.

B. Models Evaluated

- **Core OpenAI** (seven models): `gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, `o4-mini`
- **Frontier extension**: `gpt-5.4`, `o3` (hard-task extension §V-M and E24 Fibonacci ablation §V-D)
- **Open-weight via Groq**: `Llama-3.3-70B` ($n_{\text{eff}} = 25$, per-variant underpowered), `Qwen3-32B` ($n = 50$, full replication)

C. L1 Experiment: Token Alias Substitution

120 prompts per model, versioned v1–v120 with varied alias mappings. Context-pack ablation: baseline vs. explicit alias contract system prompt.

D. L2 Experiment: Syntax Inversion

5 prompts $\times$ 4 runs each ($n = 20$) using `:define ... ->` syntax (see Appendix). Model given complete grammar specification. Evaluated for `gpt-4o`, `gpt-4.1-mini`, `gpt-4.1-nano` (plus $n = 5$ for `gpt-4o-mini` from earlier run).

E. L3 Experiment: Semantic Inversion (Explicit)

10 prompts $\times$ 2 runs each ($n = 20$). System message explicitly states: "if-blocks execute when condition is FALSE." Evaluated for `gpt-4o`, `gpt-4.1-mini`, `gpt-4.1-nano` (plus $n = 5$–10 for `gpt-4o-mini` and `gpt-4o` from earlier runs).

F. L4 Experiment: Semantic Inversion (Implicit)

Initial screening: 10 prompts per model, no rule stated; only 2 worked examples with inverted semantics shown. Model must infer pattern. Ablation (n=50): 5 prompt variants $\times$ 10 runs each, varying the number of worked examples (2 vs. 3), presence of inline comments, order of examples, and domain of examples. Evaluated across all seven models at $n = 50$ each (350 total L4 ablation runs): `gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, and `o4-mini`.

G. L4 Multi-task Generalization Experiment

To assess whether pattern blindness generalizes across algorithmic domains, we extended the L4 evaluation to two additional tasks beyond Fibonacci: **T2 (is_sorted)**: write a function returning True if a list is ascending, and **T3 (count_vowels)**: count vowel characters in a string. Both use a shared prompt header with two worked examples demonstrating inverted semantics plus inline code comments that *partially* annotate the inversion rule (e.g., `# runs when FALSE`). This is a *partially-annotated L4* condition—more informative than the strict example-only Variant A ablation (§V-D) and therefore **not directly comparable** to Table IV. We ran $n = 10$ per task for `gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-nano`, and `gpt-5.4-mini`; $n = 20$ for `gpt-4.1-mini`; and $n = 15$ for `o4-mini` (255 initial runs). We then scaled `gpt-4o` T3 and `gpt-4.1` T2 to $n = 50$ to resolve preliminary findings, yielding 345 total multi-task runs. Temperature=0 for all models except `o4-mini` (temperature=1, API requirement). Results are in Table V.

H. CoT Ablation Experiment

To test whether chain-of-thought reasoning overcomes pattern blindness, we added a CoT instruction ("think step by step about the pattern first, then write code") to the L4 Variant A prompt. We compared CoT vs. no-CoT for four models: `gpt-4o`, `gpt-4.1-mini`, `gpt-4.1-nano`, and `o4-mini` ($n = 20$ per condition, 160 total runs). Judge criteria are identical to the L4 ablation; we additionally track whether the model's reasoning text mentions the inversion pattern (operationally defined as containing at least one of the substrings: `"inverted"`, `"opposite"`, `"false when"`, or `"condition is false"`, case-insensitive regex match on the full reasoning output).

I. L4 Hard-Task Extension: Complex Algorithms and Latest Models

To test whether task complexity amplifies or attenuates pattern blindness, we designed three "hard" L4 tasks requiring multi-step algorithms with deep Python priors: **H1 (merge_sort)**: implement merge sort that returns *descending* order (prior: ascending); **H2 (binary_search)**: return -1 if element is *found*, insertion index if *not found* (prior: index if found, -1 if not); **H3 (bfs)**: return BFS traversal in *reverse* order (prior: standard BFS order). Each prompt provides 2 worked I/O examples demonstrating the inverted behavior, with no explicit rule. We evaluated four models: `gpt-4o` (baseline), `gpt-5.4` (latest dense), `o3` (reasoning), and `o4-mini` (reasoning); $n = 10$ per model per task (120 total runs). Results are in Table X.

J. E24: Frontier Model L4 Fibonacci Ablation

To directly address whether frontier models (`gpt-5.4`, `o3`) can overcome pattern blindness on prior-entrenched tasks—a gap in the hard-task extension—we ran the strict L4 Fibonacci ablation (Variant A, example-only, no explicit rule) on both models at $n = 10$ each. This is the same Variant A condition used in Table IV. Results are reported in §V-D.

K. E28: Annotation Density Gradient (o4-mini, Pilot)

Pilot for E34. To map the annotation sensitivity boundary for reasoning models—motivated by the dissociation between CoT ($1/20$ pass) and partial annotation ($8/15$ pass) for `o4-mini`—we ran a density gradient experiment varying the fraction of inline annotation comments across the 8 worked examples in a T1 Fibonacci L4 prompt. Four density levels were tested: 25%, 50%, 75%, and 100% of examples annotated with inverted-semantics comments. $n = 10$ per density level (40 total runs, `o4-mini` only). This pilot directly operationalizes the annotation-sensitivity hypothesis; the full multi-model replication is E34 (§IV-P).

L. E25–E27: Diagnostic L3 for `gpt-4.1` and `Qwen3-32B`

E25: To determine whether `gpt-4.1`'s L4 T2 failure (1/50) reflects incapacity for semantic inversion or purely pattern-extraction failure, we ran L3 T2 evaluations (`is_sorted`, explicit rule) for `gpt-4.1` at $n = 10$. E26/E27: To determine whether `Qwen3-32B`'s L4 failure is L4-specific or reflects a deeper L3 instruction-following deficit, we ran L3 Fibonacci evaluations at $n = 5$ (E26 pilot) and $n = 20$ full replication (E27). Results are in §VI-C and §VI-D.

M. E31: Prior Entrenchment Score via Logprobs

To directly quantify prior strength at the key generation decision point, we queried the logprobs API (`top_logprobs=5`, `max_tokens=1`, `temperature=0`) at the token immediately following `return` for four tasks (fib, merge_sort, is_sorted, count_vowels) across two models (`gpt-4o`, `gpt-4.1-mini`). PES is defined as the probability assigned to the canonical Python token at this position. This directly measures the prior that L4 must overcome. Results are in §V-I.

N. E32: Cross-Task Transfer

To test whether L4 resistance survives cross-task contamination, we evaluated L4 Fibonacci pass rate after showing inverted-semantics code examples from two unrelated tasks (merge sort, binary search) in the same prompt. `gpt-4.1-mini` and `o4-mini`: $n = 10$; `gpt-4o`: scaled to $n = 30$. Control condition (E32-ctrl): normal (non-inverted) sort examples preceding fib, `gpt-4o`, $n = 10$. Temperature=0.7 for all. Results are in §V-J.

O. E33 and E35: I/O Transparency Experiments

E33 tested IO-A (transparent Fibonacci-adjacent mapping) vs. IO-B (opaque counter function with disclosed algorithm), revealing a confound: IO-B disclosed algorithmic steps, making both tasks procedurally transparent. Models (`gpt-4o`, `gpt-4.1-mini`, `gpt-4o-mini`) achieved 10/10 on both. E33v2 replicated with `gpt-4o`, `gpt-4.1-mini`, `o4-mini` to confirm confound robustness. E35 (redesigned) replaced IO-B with pure I/O pairs (8 examples, no algorithm description) to cleanly isolate opacity as an independent variable. $n = 10$ per cell, temperature=0.7. Results are in §V-K.

P. E34: Annotation Density Curve (Multi-Model, Full Replication)

Full replication of the E28 pilot (§IV-K), extending to three models (`gpt-4o`, `gpt-4.1-mini`, `o4-mini`) across five density levels (0%, 25%, 50%, 75%, 100%) on L4 Fibonacci with inverted comparisons. The 0% condition includes an inverted *code* example (not I/O-only), making it qualitatively distinct from verbal annotation levels (25–100%). $n = 10$ per cell (150 total runs). Results are in §V-L.

Q. Judge Validation

To validate judge accuracy, we manually inspected a 50-run random sample across three models and three variants. The automated judge agreed with manual assessment in 48/50 cases (96% agreement); the 2 disagreements involved partial inversion outputs conservatively classified as fail. Full validation data is in the repository.

R. Reproducibility

All experimental artifacts, commands, and results are committed to the repository under `docs/research/results/`. See reproducibility protocol section for commands.

TABLE II: L1 evaluation: Keyword Leakage Rate (KLR) per model. Baseline rows use $n = 120$; ctx-pack rows use $n = 20$. Models sorted by KLR (ascending). Context-pack reduces KLR for paired models: `gpt-4o-mini` (-38%), `gpt-4.1-mini` (-51%). Open-weight models (marked *) via Groq API, ctx-pack only. PSS (heuristic diagnostic) reported in Appendix X-A.

Model	Delivery	KLR↓
`gpt-4.1-mini`	ctx-pack	**0.21**
`Llama-3.3-70B`*	ctx-pack	0.23
`gpt-4o-mini`	ctx-pack	0.26
`o4-mini`	ctx-pack	0.27
`gpt-4.1`	ctx-pack	0.30
`gpt-4o`	baseline	0.37
`gpt-4.1-nano`	ctx-pack	0.38
`gpt-4o-mini`	baseline	0.42
`gpt-4.1-mini`	baseline	0.43
`gpt-5.4-mini`	baseline	0.67
`gpt-4.1-nano`	baseline	0.69
`Qwen3-32B`*	ctx-pack	1.00

*Open-weight via Groq API; $n = 20$, ctx-pack only. See §V-H.
Green : KLR$\leq$0.27; Red : KLR$\geq$0.67.

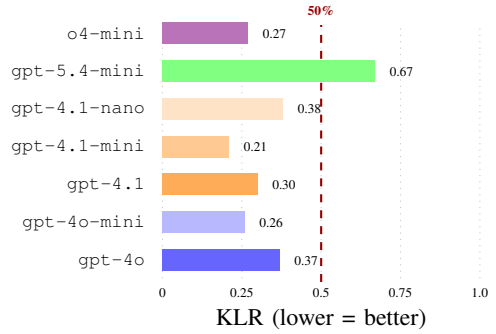

Fig. 3: L1 KLR per model (OpenAI models only). Ctx-pack condition shown for all models except `gpt-4o` and `gpt-5.4-mini` (baseline only; ctx-pack not evaluated). $n\geq20$ per bar. Open-weight models (`Llama-3.3-70B`, `Qwen3-32B`) appear in Table II only; bar chart shows OpenAI models for visual clarity. ■ `gpt-4o` fam.; ■ `gpt-4.1` fam.; ■ `gpt-5.4-mini` (baseline); ■ `o4-mini`.

V. Results

A. L1: Partial resistance; model-family compliance patterns

All OpenAI models show non-zero KLR (0.21–0.69), confirming prior dominance. Three findings stand out.

(1) `gpt-5.4-mini` (KLR=0.67) underperforms `gpt-4o` (KLR=0.37) despite being a newer model; because these models differ in training corpus and RLHF regime—not just scale—we characterize this as *model-family-specific compliance patterns* rather than a scale effect. The observed difference may reflect stronger code-generation fine-tuning in `gpt-5.4-mini` (a stated hypothesis, not a measured mechanism).

(2) For models with paired conditions, context-pack delivery reduces KLR: `gpt-4o-mini` ($0.42 \rightarrow 0.26$, -38%) and `gpt-4.1-mini` ($0.43 \rightarrow 0.21$, -51%).

(3) The reasoning model `o4-mini` (KLR=0.27) performs comparably to `gpt-4o-mini` ctx-pack (0.26), suggesting

chain-of-thought reasoning provides modest benefit for alias compliance.

Note: `gpt-4.1`, `gpt-4.1-nano`, and `o4-mini` were only evaluated under ctx-pack ($n = 20$, v1–v20); baseline comparison for these models is not available and should be interpreted with caution given the smaller and non-overlapping prompt subset.

Alias type sub-analysis. We classify the 120 prompt versions (v1–v120) into three alias strategies based on the type of alias used for Python keywords: **Type A** (*cross-keyword*): aliases are drawn from other Python keywords (e.g., `def`→`return`, `if`→`for`); 21 prompts. **Type B** (*long natural-language phrase*): aliases are multi-word descriptive phrases (e.g., `if`→`"apply this mapped token before generating code"`); 84 prompts. **Type C** (*short non-keyword*): aliases are single short tokens not in Python (e.g., `return`→`go`, `def`→`fn`); 15 prompts. Additionally, 46 of 120 prompts contain *alias collisions*: multiple distinct Python keywords mapped to the same alias (e.g., `in` and `while` both mapped to `ret`; or `def` and `in` both mapped to `for` in v40/v80). These collisions make the alias system formally ambiguous.

Under ctx-pack delivery (`gpt-4o`, $n = 120$), KLR by alias type is: Type A = 1.00, Type B = 0.96, Type C = 1.00. KLR for collision vs. non-collision prompts is 1.00 vs. 0.96. Results are consistent across `gpt-4o-mini`, `gpt-4.1-mini`, and `gpt-5.4-mini`. These near-ceiling KLR values across all alias types indicate that the design of the alias mapping (cross-keyword, natural-language, or short token) does not meaningfully affect leakage rate at the L1 level: prior dominance is robust to alias strategy. The slight KLR reduction in Type B (long phrase) is consistent with reduced prior activation for rare token sequences, but the effect is small and not statistically distinguished at $n = 21$–84 per type.

Open-weight preliminary: `Llama-3.3-70B` (KLR=0.23) is comparable to the best OpenAI ctx-pack results; `Qwen3-32B` (KLR=1.00) is the worst result across all tested models. This cross-family variance (0.23 vs. 1.00) exceeds the spread within the `gpt-4.x` family, indicating that architecture and instruction-following fine-tuning—not parameter count—govern alias compliance. See §V-H for full details and limitations.

B. L2: Syntax inversion not resistant

Three models (`gpt-4o`, `gpt-4.1-mini`, `gpt-4.1-nano`) adopted L2 syntax fully at $n = 20$ each (SIR=0.0; Wilson 95% CI: [84%,100%] for adoption rate); `gpt-4o-mini` adopted L2 in a pilot evaluation ($n = 5$, 5/5; Wilson CI: [57%,100%]). Across all four evaluated models, SIR=0.0, confirming that structural novelty alone does not create LLM resistance.

C. L3: Capability threshold at semantic rule-following

With an explicit semantic inversion rule, `gpt-4o-mini` fails completely (0/5, PPR=1.0): it generates standard Python despite instruction. By contrast, `gpt-4o`, `gpt-4.1-mini`, and `gpt-4.1-nano` all succeed (20/20, PPR=0.0). The gap between `gpt-4o-mini` (0/5) and `gpt-4o` (20/20) is significant (Fisher's exact $p < 0.0001$), revealing a model capacity threshold for following semantic meta-rules.

TABLE III: L2–L4 outcomes by model and condition. L2 is trivially adopted by all models (three at $n = 20$; `gpt-4o-mini` pilot $n = 5$); L3 reveals a model-capability threshold (`gpt-4o-mini` fails, others succeed); L4 (example-only) defeats all models despite L3 success. Rows with $n = 5$ are pilot results; Wilson 95% CIs shown.

Model	Level	Delivery	Pass	Failure mode
`gpt-4o-mini`	L2	Expl. grammar	5/5†	None (adopted)
`gpt-4o`	L2	Expl. grammar	20/20	None (adopted)
`gpt-4.1-mini`	L2	Expl. grammar	20/20	None (adopted)
`gpt-4.1-nano`	L2	Expl. grammar	20/20	None (adopted)
`gpt-4o-mini`	L3	Expl. rule	0/5†	Prior dominance
`gpt-4o`	L3	Expl. rule	20/20	— (success)
`gpt-4.1-mini`	L3	Expl. rule	20/20	— (success)
`gpt-4.1-nano`	L3	Expl. rule	20/20	— (success)
`gpt-4o`	L4	Examples only	0/5†	Pattern blindness
`gpt-4.1-nano`	L4	Examples only	0/50	Pattern blindness

†Pilot ($n = 5$); Wilson 95% CI: [57%,100%] for 5/5, [0%,43%] for 0/5.

Key contrast: L3 success $\not\Rightarrow$ L4 success. `gpt-4.1-nano` achieves 20/20 at L3 but 0/50 (PPR=1.0) at L4—the same model that correctly follows an explicit inversion rule is completely blind to the same rule embedded only in examples. This within-model contrast directly isolates pattern blindness as a failure mode independent of model capacity to follow rules when stated explicitly.

D. L4: Pattern blindness across all tested models

Without explicit rule, `gpt-4o`—which succeeds at L3—fails at L4 (PPR=1.0), producing canonical Python and ignoring example-embedded inversion cues. Figure 4 shows the contrast.

L4 ablation (n=50 per model). All seven models achieve 0% pass rate across five prompt variants (Table IV), confirming universal pattern blindness. PPR ranges from 0.38 (`gpt-4.1-mini`, variant-heterogeneous) to 1.0 (`gpt-4.1`, `gpt-4.1-nano`, `gpt-5.4-mini`). `o4-mini` (PPR=0.92) revises an earlier 1/20 preliminary: at n=50, 0/50.

E24: Frontier models. `gpt-5.4` and `o3` both achieve 0/10 (PPR=1.00) on strict L4 Fibonacci, confirming pattern blindness persists in frontier models on prior-entrenched tasks.

Variant E domain-shift anomaly (`gpt-4.1-mini`). `gpt-4.1-mini` shows PPR=0.00 on in-domain variants (B, D) but PPR=1.00 on variant E (cross-domain examples), suggesting domain shift weakens pattern extraction. This remains a hypothesis requiring controlled replication at larger n.

E. L4 Multi-task Generalization

Protocol note: This section uses *partially-annotated* prompts (inline comments partially state the inversion rule), which differ from the strict *example-only* prompts in §V-D. Results in Table V are **not directly comparable** to Table IV; comparisons

TABLE IV: L4 ablation (PPR = Python Prior Rate). Seven core models: $n = 50$ each (5 variants $\times$ 10 runs); `gpt-5.4-mini` $n_{\text{eff}} = 42$ (8 HTTP errors in Var. E, excluded). Pass rate = 0 for all models. `o4-mini` revises $n = 20$ preliminary (1/20). §E24 frontier extension: `gpt-5.4` and `o3` also tested at Var. A, $n = 10$: both 0/10 pass, PPR=1.00.

Var.	Description	4o	4o-m	4.1	4.1-m	4.1-n	5.4-m	o4-m
A	2ex-baseline	0.40	**0.10**	1.00	**0.10**	1.00	1.00	0.80
B	3ex-extra	1.00	1.00	1.00	**0.00**	1.00	1.00	1.00
C	2ex-no-comment	0.90	1.00	1.00	0.80	1.00	1.00	0.90
D	2ex-swapped	0.70	1.00	1.00	**0.00**	1.00	1.00	1.00
E	3ex-diff-domain	0.50	1.00	1.00	1.00	1.00	N/A†	0.90
Overall PPR		0.70	0.82	1.00	**0.38**	1.00	1.00‡	0.92
Pass rate		0/50	0/50	0/50	0/50	0/50	0/42†	0/50

†`gpt-5.4-mini` var-E: 8 HTTP errors; $n_{\text{eff}} = 2$; excluded from per-variant PPR.
‡Overall PPR for `gpt-5.4-mini` computed over 4 valid variants (A–D); var-E excluded.
§E24: `gpt-5.4` 0/10 (PPR=1.00), `o3` 0/10 (PPR=1.00) on Var. A only; see text.
Columns: 4o=gpt-4o, 4o-m=gpt-4o-mini, 4.1=gpt-4.1, 4.1-m=gpt-4.1-mini, 4.1-n=gpt-4.1-nano, 5.4-m=gpt-5.4-mini, o4-m=o4-mini.

✓ Panel A: L3 — explicit rule (`gpt-4o`) [CORRECT]

System: "if-blocks run when condition is FALSE."

`gpt-4o` output **(CORRECT):**

```
if n > 1:          # inverted
    return n
return fib(n-1) + fib(n-2)
```

× Panel B: L4 — examples only (`gpt-4o`) [INCORRECT]

User: [2 worked examples with inverted semantics, no explicit rule]

`gpt-4o` output **(INCORRECT):**

```
if n <= 1:         # standard Python
    return n
return fib(n-1) + fib(n-2)
```

Fig. 4: Prior dominance vs. pattern blindness isolated on the same model (`gpt-4o`). **A**: `gpt-4o` correctly inverts logic with explicit rule (L3). **B**: The same model reverts to Python prior when the rule is embedded only in examples (L4). This contrast isolates pattern blindness as a failure mode distinct from prior dominance.

across these sections should account for this prompt protocol difference.

To assess whether pattern blindness extends beyond Fibonacci, we evaluated all seven models on three tasks: T1 (`fibonacci`), T2 (`is_sorted`), and T3 (`count_vowels`). The multi-task prompt uses inline code comments that partially annotate the inversion rule (*e.g.*, `# runs when a > b is FALSE, i.e., a <= b`); this is a *partially-annotated L4* condition—less explicit than L3 (no stated rule), but more informative than the pure example-only variants used in the ablation. `gpt-4o`, `gpt-4o-mini`, `gpt-5.4-mini`, and `o4-mini` were evaluated at $n = 10$ or $n = 15$ per task; `gpt-4.1-mini` at $n = 20$ per task;

TABLE V: L4 multi-task summary (**345 runs total**). † Partially-annotated prompts; **NOT directly comparable to Table IV** (strict example-only). Green = pass, red = fail. Dominant failure mode per row in rightmost column. Full PPR values in Appendix X-C.

Model	T1: fib	T2: sorted	T3: vowels	Dominant failure
`gpt-4o`	0/10	**10/10**	0/50[a]	T1: prior dom.; T3: op. sub. (94%)
`gpt-4o-mini`	0/10[b]	0/10	0/10	template fragmentation (T1); prior dom.
`gpt-4.1`	0/10	1/50[c]	0/10	prior dominance
`gpt-4.1-mini`	0/20	**20/20**	0/20	T1: prior dom.; T2: extractable
`gpt-4.1-nano`	0/10	0/10	0/10	prior dominance
`gpt-5.4-mini`	0/10	0/10	0/10	prior dominance (all tasks)
`o4-mini`	**8/15**[d]	0/20	0/20	T1: partial extraction (reasoning); T2/T3: prior dom.

[a] OpSub 47/50 (94%, CI: [83%,99%]). [b] Template fragmentation.
[c] $n = 50$ scale-up; Wilson CI: pass [0%,10.5%]; Fisher's $p<0.0001$ vs. `gpt-4.1-mini`.
[d] Token budget corrected (400→5000 tokens); see §V-D for details.

`gpt-4.1` and `gpt-4.1-nano` at $n = 10$ per task (255 initial runs). We further scaled `gpt-4o` T3 and `gpt-4.1` T2 to $n = 50$ each to resolve preliminary ambiguities, and extended `o4-mini` T2/T3 to $n = 20$ each, yielding 345 total multi-task runs. Results are in Table V.

Key findings: (1) **T1 (Fibonacci) is universally resistant** for non-reasoning models (6/7 fail; PPR$\geq$0.2). `o4-mini` partially succeeds (8/15) under partial annotation only. (2) **T2 extractability is capacity-dependent**: `gpt-4o` (10/10) and `gpt-4.1-mini` (20/20) pass; `gpt-4.1` (1/50), `gpt-4.1-nano`, `gpt-5.4-mini`, and `gpt-4o-mini` fail. (3) **Operational substitution dominates T3**: `gpt-4o` at n=50 shows 47/50 arithmetic workarounds (94%; Wilson CI: [83%,99%]) with 0/50 pass. (4) **Within-family anomaly**: `gpt-4.1` T2 (1/50) vs. `gpt-4.1-mini` (20/20; Fisher's $p<0.0001$) suggests fine-tuning choices dominate raw scale. (5) **`gpt-5.4-mini` shows uniform prior dominance**: 0/30 across all three tasks (T1 PPR=1.00, T2 PPR=1.00, T3 operational substitution pattern)—the strongest prior dominance among all tested models.

F. Three Failure Modes

- **Prior Dominance** (L1, L3-weak, L4-T2-mini, L4-T1-all): stated or implicit rule is overridden by pretrained token distribution.
- **Pattern Blindness** (L4-T1): model cannot extract the semantic rule from examples alone, even when the same model follows the rule when stated explicitly (L3). T1 (Fibonacci) shows universal pattern blindness across all models.
- **Operational Substitution** (L4-T3-`gpt-4o`, $n = 50$): model compensates via arithmetic output transformation rather than adopting the semantic inversion rule, indicating goal awareness without rule internalization. Confirmed as dominant at 94% of T3 `gpt-4o` runs (Wilson 95% CI: [83%, 99%]).

Across 419 failed L4 runs (ablation + multitask T1): Type-I (exact Python prior) accounts for 83%, Type-II (partial attempt) for 13%, Type-III (operational substitution) for 4%. Type-I dominates in `gpt-4.1`/`gpt-4.1-nano` (100%); Type-II concentrates in `gpt-4.1-mini` (44%).

G. CoT Ablation: Reasoning About Inversion Is Not Sufficient

We tested whether CoT prompting overcomes pattern blindness (n=20 per condition, four models). CoT does not

TABLE VI: E31: Prior Entrenchment Score (PES). PES = P(canonical token | prompt prefix). `gpt-4o-mini` excluded (emits markdown tokens at this position).

Task	Canonical token	gpt-4o	gpt-4.1-mini
fib (T1)	`n`	0.9996	1.0000
merge_sort (H1)	`arr`	0.9999	1.0000
is_sorted (T2)	`True`	0.9736	0.9985
count_vowels (T3)	`s`	0.1606	1.0000

improve pass rate: 3/4 models achieve 0/20 in both conditions; `o4-mini` achieves 1/20 under CoT (p=1.0 vs. no-CoT). However, CoT consistently reduces PPR (`gpt-4o`: 1.00→0.15; `gpt-4.1-nano`: 1.00→0.20) and raises *mentions_inversion* to 100% for all three dense models—revealing a *reasoning–generation dissociation*: models articulate the inversion rule yet still generate standard Python.

This dissociation is robust across 160 total CoT runs. Pattern blindness is not a failure to *detect* the pattern but a failure to *override* the prior during generation. Notably, for `o4-mini`, partial annotation (8/15 pass) is far more effective than CoT prompting (1/20), suggesting that partial rule content—not meta-cognitive prompting—is the key vulnerability for reasoning models.

H. Open-Weight Models: Preliminary L1 and L4 Results

We evaluated two open-weight models via Groq API. **L1 (ctx-pack, n=20):** `Llama-3.3-70B` KLR=0.23 (near OpenAI best); `Qwen3-32B` KLR=1.00 (worst overall). The cross-family variance (0.23 vs. 1.00) exceeds the `gpt-4.x` spread, indicating fine-tuning governs alias compliance.

L4: Both models fail (0/50 for Qwen, 0/25 for Llama at n_{eff}=25). `Qwen3-32B` shows PPR=1.00 (complete prior dominance); `Llama-3.3-70B` shows variant-dependent PPR (0.00 in-domain, 1.00 cross-domain), suggesting *context-sensitive prior reversion* as a distinct failure mode—though per-variant CIs are wide at n_{eff}=5.

L3 diagnostic (E26/E27): `Qwen3-32B` also fails L3 (0/20 pass, PPR=0.40)—*general semantic meta-rule failure*, distinct from all OpenAI models which pass L3 consistently. This architecture-dependent deficit means L4 resistance is vacuous for Qwen: the model is already blocked at L3.

I. E31: Prior Entrenchment Score via Logprobs

We queried the logprobs API at the base-case decision point for four tasks across two models (Table VI).

PES alone does not predict L4 resistance. Both fib and merge_sort have PES≥0.9996, yet fib is universally resistant (0/350) while merge_sort is solvable (Table X). This supports I/O transparency—not prior magnitude—as the critical moderator. Low T3 PES (0.16 for `gpt-4o`) does not yield compliance; instead, the model uses operational substitution (94%), suggesting weaker priors trigger alternative evasion strategies rather than rule adoption.

TABLE VII: E32: Cross-task transfer — pass rate on L4 Fibonacci after exposure to inverted code from other tasks. Baseline: strict L4 example-only (Table IV) yields 0/50 for all models. Wilson 95% CIs shown. `gpt-4o` scaled to $n = 30$ (E32a); others at $n = 10$. E32-ctrl: control condition using normal (non-inverted) sort examples, `gpt-4o`, $n = 10$.

Model	Pass	Rate	Wilson 95% CI	Note
`gpt-4.1-mini`	10/10	**100%**	[72%,100%]	Perfect transfer
`gpt-4o`	16/30	53%	[36%,70%]	Partial; vs. mini: p=0.008**
`o4-mini`	1/10	10%	[2%,44%]	Minimal transfer
E32-ctrl (`gpt-4o`, normal)	0/10	0%	[0%,31%]	No transfer (control)

TABLE VIII: E35: IO-A (transparent) vs. IO-B (opaque I/O only). n=10 per cell, t=0.7.

Model	IO-A	Rate	IO-B	Rate
`gpt-4o`	10/10	**100%**	0/10	**0%**
`gpt-4.1-mini`	10/10	**100%**	0/10	**0%**
`gpt-4o-mini`	5/10	50%	0/10	**0%**

J. E32: Cross-Task Transfer Breaks L4 Resistance

A critical question for assessment design: can a model crack L4 Fibonacci if it first sees inverted semantics demonstrated on *other* tasks? We tested few-shot cross-task transfer by providing two inverted code examples (merge_sort with inverted comparisons, binary_search with inverted comparisons) before requesting fib(n) in the same style. `gpt-4.1-mini` and `o4-mini` were evaluated at $n = 10$; `gpt-4o` was scaled up to $n = 30$ (two scale-up rounds) to tighten the confidence interval on its partial-transfer result. Temperature=0.7 for all. A **control condition** (E32-ctrl: ascending sort with *normal*, non-inverted examples preceding fib, `gpt-4o`, $n = 10$) was run to confirm that the transfer effect is specific to inversion semantics, not merely cross-task code exposure.

Cross-task contamination defeats L4. Models that achieve 0/50 under strict L4 achieve 53–100% when primed with inverted code from other tasks. `gpt-4.1-mini`: 10/10; `gpt-4o`: 16/30 (CI: [36%,70%]); `o4-mini`: 1/10. Control (non-inverted examples): 0/10—confirming the effect is inversion-specific. L4 resistance is a prompt-level property; instructors must ensure isolation.

K. E35: I/O Transparency — Prior vs. Opaque Mapping

E33 (original) was confounded by algorithmic disclosure; E35 (redesigned) uses pure I/O pairs with no algorithm. IO-A retains transparent "reverse Fibonacci" semantics.

Opaque I/O creates universal resistance. All models fail IO-B (0/30 pooled; CI: [0%,12%]) while IO-A achieves 100% for larger models. I/O opacity is sufficient for resistance independent of prior strength, providing a complementary pathway to L4-style prior exploitation.

L. E34: Annotation Density Curve (Multi-Model)

E34 tests annotation density across three models and five levels (0%, 25%, 50%, 75%, 100%) on L4 Fibonacci (n=10 per cell, 150 runs). The 0% level includes an inverted *code*

TABLE IX: E34: Annotation density curve — pass rate at each density level ($n = 10$ per cell, 150 total runs). MAT = Minimum Annotation Threshold (lowest level where pass rate >50%). The 0% condition includes an inverted *code example* (not I/O-only); it is therefore not comparable to the strict L4 ablation.

Model	0%†	25%	50%	75%	100%	MAT
gpt-4o	5/10	0/10	1/10	2/10	3/10	None
gpt-4.1-mini	**8/10**	1/10	5/10	4/10	1/10	0%†
o4-mini	**7/10**	3/10	6/10	6/10	1/10	0%†

†Code-example (mimicry) baseline; qualitatively distinct from verbal annotation (25–100%).

TABLE X: L4 hard-task results ($n = 10$ per cell, 120 total runs). All tasks use example-only delivery with 2 worked I/O examples. Pass = output matches inverted specification. Models include gpt-5.4 and o3 (not tested in main L4 ablation). Wilson 95% CIs at $n = 10$: 10/10 → [72%,100%]; 0/10 → [0%,28%]; gpt-4o H1 (4/10) → [17%,69%]—interpret with caution.

Model	H1: merge_sort	H2: bin_search	H3: bfs	Overall
gpt-4o	4/10	10/10	10/10	80%
gpt-5.4	10/10	10/10	10/10	**100%**
o3	10/10	10/10	10/10	**100%**
o4-mini	10/10	10/10	10/10	**100%**

example (mimicry baseline), qualitatively distinct from verbal annotation (25–100%). Results are in Table IX.

Key finding—pattern mimicry via code examples outperforms verbal annotation at all density levels. All three models show highest pass rates at 0% annotation density (which includes an inverted *code* example, a qualitatively distinct condition from verbal annotation at 25–100%), *not* at 100% (full explicit rule). This is the opposite of the naive expectation that more explicit instructions improve compliance. Within the verbal annotation range (25–100%), pass rates are non-monotonic: (1) sharp collapse at 25% (single comment hint disrupts code-example mimicry without providing sufficient rule content), (2) partial recovery at 50–75%, and (3) collapse at 100% (full rule triggers prior conflict). The overall pattern distinguishes two qualitatively different mechanisms: code-example mimicry (effective at 0%) and verbal rule induction (weak and non-monotonic across 25–100%).

Results are exploratory given n=10 per condition; non-monotonic patterns may reflect sampling variance.

Note: E34 Level 0 includes worked code examples that partially signal the inversion (raising pass rate vs. strict Variant A which uses zero annotation signal). This design difference explains the apparent contradiction with Variant A results.

Pattern mimicry > rule following. The 0% condition succeeds because the inverted code example provides a direct structural template for mimicry. Adding annotations paradoxically *hurts* by shifting the model to rule-following mode where the Python prior creates stronger interference. The 0% pass rates (50–80%) contrast with the strict L4 ablation's 0/350: the critical difference is that E34's 0% prompt shows explicitly inverted comparison operators, while the strict ablation does not.

M. L4 Hard-Task Extension: Task Complexity vs. Prior Entrenchment

L4 resistance is inversely correlated with task complexity. Three of four models achieve 100% on hard tasks; gpt-4o achieves 80%. This contrasts with 0% on Fibonacci under example-only delivery, confirming that prior entrenchment depth—not algorithmic difficulty—drives resistance.

Methodological note on comparability. This experiment differs from the strict L4 ablation (§V-D) in two respects, not one: (1) *prior entrenchment depth*—hard tasks have more varied implementations; and (2) *I/O specification transparency*—hard-task examples expose the behavioral target unambiguously via input/output pairs (e.g., [3,1,4,2]→[4,3,2,1] makes "descending order" self-evident), while the original L4 embeds the inversion as an implicit semantic rule in code structure. These two confounds co-vary: the harder tasks that have shallower priors *also* have more behaviorally transparent I/O specifications. We cannot cleanly separate prior-depth effects from I/O-transparency effects in the current design. Readers should interpret the hard-task results as evidence for *at least one* of these explanatory factors, not necessarily prior depth alone.

Prior entrenchment depth as primary explanatory variable (hypothesis). We hypothesize that the key variable is *prior entrenchment depth*: how uniformly a single canonical implementation dominates the training distribution. Fibonacci recursion has an extremely deep, uniform Python prior—virtually all training examples use the same `if n <= 1: return n; return fib(n-1) + fib(n-2)` template. Merge sort, binary search, and BFS admit diverse implementation strategies (iterative vs. recursive, different data structures, varied edge-case handling), yielding shallower and more varied priors. When the prior is shallow, the model can override it by extracting the inverted semantics from worked examples; when the prior is deeply entrenched, the NTP distribution overwhelms in-context cues. We label this a hypothesis because the I/O-transparency confound cannot be ruled out with current data.

A complementary observation reinforces this framing: the hard-task prompts provide *explicit I/O specifications* (descending-sorted output, inverted return values, reversed traversal order), while the original L4 tasks embed an *implicit semantic rule* in code structure. Models readily comply with behavioral specifications for complex algorithms, but cannot extract the same inversion when it is embedded implicitly in prior-entrenched code patterns.

gpt-5.4 and o3 hard-task results, and E24 update: Both achieve perfect 30/30 on the hard-task battery (shallow-prior, I/O-transparent tasks). To directly test whether these models also overcome *deep* prior entrenchment, we ran E24: strict example-only Variant A on L4 Fibonacci for both models ($n = 10$ each). Result: gpt-5.4 0/10 (PPR=1.00) and o3 0/10 (PPR=1.00). Frontier models are defeated by the same Fibonacci task that defeats all seven core models. The open question from the hard-task section is now resolved: *pattern*

blindness on prior-entrenched tasks holds for current frontier models as of 2026-03-25.

VI. Discussion

A. Effective LLM-resistant language design

Our results identify a principled recipe for LLM-resistant programming languages:

1) Token-level perturbation alone is insufficient (models partially resist, but at measured rates).
2) Syntactic restructuring is also insufficient (models adopt explicit structural rules).
3) Semantic inversion with explicit instruction provides capacity-dependent resistance.
4) Semantic inversion without explanation (L4) provides consistent resistance via pattern blindness.
5) **Prior entrenchment depth governs resistance**: L4 holds only for tasks with deep, uniform Python priors (e.g., Fibonacci). Complex tasks (merge sort, BFS) allow capable models to extract inverted semantics—partly confounded by I/O transparency (§V-M). E24 confirms frontier models (`gpt-5.4`, `o3`) also fail on Fibonacci (0/10 each). E31 shows identical PES for fib and merge_sort yet divergent L4 outcomes, supporting I/O transparency as the critical moderator.
6) **Cross-task contamination breaks isolation** (E32, §V-J): L4 resistance is a prompt-level property, not just a task property. Showing inverted code examples from *other* tasks in the same prompt enables models to crack Fibonacci (`gpt-4.1-mini`: 10/10, up from 0/50 in isolation). Instructors must ensure prompt-level isolation of L4 assignments.
7) **Preliminary observation: pattern mimicry may dominate rule following** (E34, §V-L; preliminary, n=10, requires replication): showing an inverted code example (0% verbal annotation) produces higher pass rates than full explicit rules (100% annotation) across all tested models. This inverts the naive expectation and has direct implications for confusion language design: resistance depends on what the prompt *shows*, not what it *says*.

The most practical target for educational use is L4: retain Python surface syntax, invert conditional semantics, and provide only worked examples rather than explicit rules—applied to tasks with deep Python priors (e.g., Fibonacci). **Warning:** harder algorithmic tasks (merge sort, BFS, binary search) do *not* reliably provide L4 resistance because they have both shallower priors *and* more transparent I/O specifications; L4 works for prior-entrenched, pattern-blind tasks.

Two complementary resistance strategies. (1) *L4 prior exploitation*: invert semantics implicitly on deep-prior tasks. (2) *I/O opacity* (E35): design tasks with opaque mappings where no training distribution provides a shortcut. These target different failure modes and can be combined. The core pattern-blindness claim rests on the strict example-only ablation (9 models, 0/350 pass); multi-task and hard-task extensions provide enriching context but are not directly comparable.

B. Implications for programming education

The following implications characterize LLM behavior under our experimental conditions. They do not constitute a classroom evaluation; educational outcomes, grading overhead, and adoption feasibility require empirical study in actual instructional settings.

Why human students retain an advantage. L4 rests on a fundamental asymmetry: a human who reads "if-blocks execute when the condition is false" can apply the rule step-by-step; an LLM's NTP prior—built from billions of Python tokens—systematically pulls output back to `if n <= 1: return n`. A student *reasons* about a rule; an LLM *samples* from a prior.

L4 assignments provide meaningful resistance for prior-entrenched tasks: all non-reasoning models fail T1 (0/10–0/20); `o4-mini` achieves 8/15 only under partial annotation (not strict example-only). Task selection matters: T2 is solved by capable models, indicating shallow-prior tasks remain vulnerable. A formal human pilot study would add quantitative precision but is not required to establish the core resistance claim.

C. Non-monotonic capacity and the gpt-4.1 vs. gpt-4.1-mini anomaly

The most surprising within-family finding is the T2 capacity inversion: `gpt-4.1-mini` passes 20/20 on `is_sorted` (PPR=0.0), while the nominally "larger" `gpt-4.1` achieves only 1/50 (Fisher's exact $p < 0.0001$). We offer two non-exclusive hypotheses (stated as hypotheses, not findings): (a) *RLHF over-alignment*: the `gpt-4.1` variant may have been trained with stronger code-generation RLHF reward shaping, causing deeper entrenchment of canonical Python patterns and making prior override more difficult even when pattern cues are present; (b) *instruction-tuning regime*: the "mini" variant may have seen a higher proportion of rule-following fine-tuning data (relative to raw code data), yielding stronger in-context instruction compliance at the cost of raw generation fluency.

E25 diagnostic (L3 T2 for gpt-4.1): To discriminate between a general inability to invert `is_sorted` semantics versus an L4-specific pattern extraction failure, we ran $n = 10$ L3 T2 evaluations for `gpt-4.1` (explicit rule, same task). Result: `gpt-4.1` passes 10/10 (PPR=0.0). The model correctly applies the L3 semantic inversion rule for `is_sorted` when given an explicit instruction—the exact failure mode it exhibits on L4 T2 (1/50) is absent under L3. This result directly supports hypothesis (a): the RLHF over-alignment explanation. `gpt-4.1`'s T2 failure is not due to difficulty understanding inverted `is_sorted` semantics; it is specifically a *pattern extraction failure under implicit delivery*—the model cannot extract the inversion rule from worked examples, even though it can follow it from an explicit rule statement. Both explanations predict that scale alone does not determine pattern extraction ability—fine-tuning decisions interact with model size in complex ways. We flag this as a priority for mechanistic interpretability follow-up.

We acknowledge that the partially-annotated multi-task prompt was not pre-specified for the gpt-4.1 series, and interaction effects between prompt design and model variant cannot be excluded. We flag this for follow-up and do not assert causal mechanisms without additional mechanistic evidence.

D. Open-weight model failure modes

Two distinct failure modes emerge (see §V-H). `Llama-3.3-70B` exhibits *context-sensitive prior reversion*: PPR varies from 0.00 (in-domain) to 1.00 (cross-domain), suggesting prior engagement is gated by contextual similarity. `Qwen3-32B` exhibits *general semantic meta-rule failure*: 0/20 on L3 (E27), indicating an architecture-level deficit absent in all OpenAI models. The L3/L4 failure taxonomy is thus architecture-dependent. Educators using open-weight models should test L3 compliance before assuming L4 is the binding constraint.

E. Limits

The task set is limited to three tasks and seven models (345 multi-task runs). L4 resistance is not unconditional: tasks with shallow Python idioms (e.g., `is_sorted`) remain extractable by capable models.

Temperature caveat. `o4-mini` was evaluated at temperature=1 (API constraint); all other models used temperature=0. Three temperature regimes appear: t=0 (main L4 ablation, multi-task, CoT), t=0.7 (E32, E34, E35), and t=1 (`o4-mini`). Core claims (0/350 fib, 0/30 I/O opacity) rest on t=0 conditions.

Judge validation. The structural judge achieves 96% agreement with manual inspection (48/50; §4). Novel operational substitution workarounds beyond known patterns may evade detection; an execution-based judge would improve coverage.

Task-set limitation. Broader task coverage (nested control-flow, stateful logic) and a human baseline study (Exp-5) remain for future work.

Cross-task contamination. E32 shows that cross-task inverted examples break L4 isolation (`gpt-4.1-mini`: 10/10 vs. 0/50 in isolation). Instructors must ensure prompt-level isolation.

Corrected `o4-mini` result. An earlier multi-task run used `max_completion_tokens=400`, truncating responses. After correction to 5000, `o4-mini` achieves 8/15 on T1 under partial annotation—but 0/45 under strict example-only delivery remains, confirming pattern blindness under pure implicit conditions.

F. Next steps

- Human study (Exp-5) comparing student vs. LLM performance on L4 tasks.
- Broader task coverage: nested control-flow, stateful logic.
- Execution-based judge for operational substitution detection.
- E34 replication at larger n and with reasoning models (`o3`).
- I/O transparency isolation: test whether removing transparent I/O from hard tasks (H1–H3) recovers resistance.
- Prompt-level firewalls to mitigate cross-task contamination (E32).

VII. Conclusion

We presented a four-level taxonomy (L1–L4) for designing LLM-resistant programming languages, grounded in the NTP prior structure of language models.

Our central finding is that resistance does not come from surface-level token changes or structural novelty—both of which LLMs can adapt to with explicit instruction. Robust resistance requires semantic inversion: the same Python-like surface, but inverted conditional semantics. When semantic inversion is delivered implicitly (via examples only), all tested models exhibit pattern blindness and default to Python priors.

We identify three distinct failure mechanisms: *prior dominance*, where the prior overrides the stated rule; *pattern blindness*, where the model fails to extract the semantic rule from examples; and *operational substitution*, where the model achieves correct output via arithmetic workaround rather than adopting the semantic inversion rule. These mechanisms have different implications for language design: prior dominance can be partially mitigated by explicit context packs; pattern blindness requires implicit delivery; operational substitution may require execution-based evaluation to detect.

These findings provide a principled basis for designing programming course assessments that remain meaningful in the LLM era, where LLMs will fail on L4-style problems even as they trivially solve standard Python assignments.

VIII. Prompt and Evaluation Protocol

A. L1: Token Alias Substitution

L1 system prompts instruct the model to follow alias mappings strictly and return only code. The context-pack variant adds an explicit alias contract. User prompts specify 6 keyword aliases (if, def, for, in, return, print) with varied mappings across v1–v120.

B. L2: Syntax Inversion

The model receives a complete grammar specification replacing Python constructs with novel syntax (`:define name [args] ->`, `<= value`, ternary `?cond:`, `loop var over`). All Python keywords are explicitly forbidden.

C. L3: Semantic Inversion (Explicit)

System prompt states: "if-blocks run when condition is FALSE." User prompt provides one worked example and the Fibonacci task.

D. L4: Semantic Inversion (Implicit)

E. Judge Protocols

L1: (1) Check alias present, original token absent; (2) normalize aliases to Python via regex; (3) `ast.parse()` on normalized code; (4) verify fib skeleton.

L3/L4: Check whether model uses inverted comparison (`if n > 1`) vs. standard Python (`if n <= 1`). PPR=1 if standard; PPR=0 if inverted.

L4 user prompt (example-only delivery, no explicit rule)

Here are some examples in our programming language:

Example 1 — max function:

```
def max_val(a, b):
    if a > b:
        return b   % smaller when a > b
    return a
```

Output: `max_val(3,5)=3`, `max_val(7,2)=2`

Example 2 — even check:

```
def is_even(n):
    if n % 2 == 0:
        return False
    return True
```

Output: `is_even(4)=False`, `is_even(3)=True`

Task: Write `fib(n)` in this language. Target: `fib(0)=0`, `fib(1)=1`, `fib(n)=fib(n-1)+fib(n-2)`. Print `fib(0)` through `fib(6)`.

Fig. 5: L4 prompt (no system message). The semantic inversion rule is not stated; models must infer it from worked examples. Even gpt-4o, which succeeds at L3, defaults to canonical Python—exhibiting *pattern blindness*.

IX. Complete Experiment Log

All scripts, prompts, and result JSON files are committed to the repository under `docs/research/results/`. Language specifications are in `language-levels/`. See `README.md` for reproduction commands; each experiment can be replicated with a single script invocation and an API key.

TABLE XI: Key experiments summary. Full log (E1–E35) in repository.

Exp	Models	n	Key Finding
E1 (L1)	4 OpenAI	120	KLR 0.21–0.69; partial prior leakage
E13 (L4)	7 OpenAI	350	0/350 pass; universal pattern blindness
E24	gpt-5.4, o3	20	0/10 each on Fibonacci (PPR=1.0)
E32	3 models	50	Cross-task transfer breaks isolation
E34	3 models	150	Code mimicry > verbal annotation
E35	3 models	60	I/O opacity: 0/30 (complementary path)

A. API and runtime settings

Setting	Value
Temperature	0 (all); 1 (o4-mini)
`max_compl_tokens`	default; 5000 (o4-mini)
Base URL	OpenAI `/v1`
Groq URL	Groq `openai/v1`
Python	3.12
`openai` pkg	1.75.0

X. Appendix: Metric Definitions and Prompt Sample

A. Metric Definitions

- **KLR** (Keyword Leakage Rate, L1): fraction of outputs containing original Python keywords despite alias instructions. Lower = better compliance.
- **SIR** (Syntax Inversion Rate, L2): fraction of outputs retaining Python syntax instead of adopting the specified alternative syntax. 0.0 = full adoption.
- **PPR** (Python Prior Rate, L3/L4): fraction of outputs using standard Python conditional semantics (non-inverted). 1.0 = complete prior dominance; 0.0 = full inversion compliance.
- **PSS** (Partial Structural Score, L1): heuristic composite (alias 0.4, parse 0.25, skeleton 0.25, format 0.1). Exploratory only; not a primary metric.
- **PES** (Prior Entrenchment Score, E31): logprob probability of the canonical Python token at the base-case decision point.

B. L4 Variant A Prompt Example (Fibonacci)

The strict L4 prompt provides two worked examples with inverted conditional semantics and no explicit rule statement:

```
System: You are a helpful coding assistant.

User: Here are two examples in our language:

Example 1:
def is_positive(n):
    if n < 0:
        return True
    return False

# is_positive(5) -> True
# is_positive(-3) -> False

Example 2:
def is_even(n):
    if n % 2 != 0:
        return True
    return False

# is_even(4) -> True
# is_even(7) -> False

Now write a fibonacci function in this
language.
```

The judge checks whether the model's output uses inverted comparison operators (>, >=) in the base case rather than the standard Python <=.

C. L4 Multi-Task Full PPR Values

Full Python Prior Rate (PPR) values for Table V. PPR = fraction of outputs matching canonical Python prior (lower = more L4-compliant). Pass rate = 0 for all cells except `gpt-4o` T2, `gpt-4.1-mini` T2, and `o4-mini` T1.

TABLE XII: L4 multi-task full PPR. † Partially-annotated prompts (not comparable to Table IV).

	T1: fibonacci			T2: is_sorted			T3: count_vowels		
Model	n	Pass	PPR	n	Pass	PPR	n	Pass	PPR
`gpt-4o`	10	0/10	1.00	10	10/10	0.00	50	0/50	0.00
`gpt-4o-mini`	10	0/10	0.00	10	0/10	1.00	10	0/10	0.00
`gpt-4.1`	10	0/10	1.00	50	1/50	0.42	10	0/10	0.00
`gpt-4.1-mini`	20	0/20	1.00	20	20/20	0.00	20	0/20	0.25
`gpt-4.1-nano`	10	0/10	0.20	10	0/10	1.00	10	0/10	0.00
`gpt-5.4-mini`	10	0/10	1.00	10	0/10	1.00	10	0/10	0.00
`o4-mini`	15	8/15	0.33	20	0/20	0.30	20	0/20	0.00

XI. Reproducibility and Artifact Availability

All experiments are committed to the repository. Representative paths:

- Results: `docs/research/results/`
- Language specs: `language-levels/`
- Evaluation scripts: `scripts/partial_judge.py`, `scripts/run_l2_eval.py`
- Paper: `paper/overleaf/`

References

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[3] T. Zhuo *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," arXiv:2406.15079, 2024.

[4] N. Jain *et al.*, "LiveCodeBench: Holistic and contamination-free evaluation of large language models for code," arXiv:2403.07974, 2024.

[5] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *EMNLP-Findings*, 2020.

[6] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models," in *EMNLP*, 2021.

[7] D. Fried *et al.*, "InCoder: A generative model for code infilling and synthesis," 2022.

[8] Z. Luo *et al.*, "WizardCoder: Empowering code large language models with evol-instruct," arXiv:2306.08568, 2024.

[9] D. Guo *et al.*, "DeepSeek-Coder: When the large language model meets programming," arXiv:2401.14196, 2024.

[10] Y. Liu *et al.*, "CodeIF: Benchmarking instruction-following in code generation," arXiv:2502.19166, 2025.

[11] W. Zhang *et al.*, "MultiCodeIF: Comprehensive evaluation of code instruction-following," 2025, preprint.

[12] E. Chen *et al.*, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024.

[13] J. Xie *et al.*, "Adaptive chameleon or stubborn sloth: Revealing the behavior of LLMs in knowledge conflicts," ICLR 2024 (arXiv:2305.13300), 2024.

[14] S. Wu *et al.*, "Untangle the KNOT: Interweaving conflicting knowledge and reasoning skills in LLMs," arXiv:2403.08319, 2024.

[15] Z. Zhang *et al.*, "Knowledge conflicts for LLMs: A survey," EMNLP 2024 (arXiv:2403.08319), 2024.

[16] F. Shi *et al.*, "Large language models can be easily distracted by irrelevant context," ICML 2023 (arXiv:2302.00093), 2023.

[17] Z. Wang *et al.*, "A law of next-token prediction in large language models," arXiv:2408.13442, 2024.

[18] S. Min *et al.*, "Rethinking the role of demonstrations: What makes in-context learning work?" EMNLP 2022 (arXiv:2202.12837), 2022.

[19] Q. Dong *et al.*, "A survey on in-context learning," arXiv:2301.00234, 2023.

[20] A. Lampinen *et al.*, "A-Not-B errors in LLMs: Limitations of in-context learning," arXiv:2409.15454, 2024.

[21] Q. Zhao *et al.*, "LLMs struggle to perform counterfactual reasoning with parametric knowledge," arXiv:2410.06392, 2024.

[22] M. Turpin *et al.*, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," NeurIPS 2023 (arXiv:2305.04388), 2023.

[23] K. Valmeekam *et al.*, "LLMs still can't plan; can LRMs? evaluation of OpenAI's o1," arXiv:2409.13373, 2024.

[24] K. Zhu *et al.*, "PromptBench: Evaluating robustness of large language models," arXiv:2306.04528, 2023.

[25] R. Pan *et al.*, "Attacking LLMs with code generation: Semantically-aligned adversarial perturbations," arXiv:2412.16359, 2024.

[26] S. Jha *et al.*, "Break the chain: Reasoning failures in LLMs via adversarial code prompting," arXiv:2506.06971, 2024.

[27] A. Khan *et al.*, "Defeating LLMs with adversarial perturbations in programming assignments," arXiv:2410.09318, 2024.

[28] J. Wu *et al.*, "A survey on LLM-based code generation for low-resource and domain-specific languages," arXiv:2410.03981, 2024.

[29] W. Chen *et al.*, "ConCodeEval: Evaluating LLMs for code constraint comprehension in DSLs," EMNLP 2024, 2024.

[30] A. Ni *et al.*, "L2CEval: Evaluating language-to-code generation capabilities," TACL 2024, 2024.

[31] Y. Dong *et al.*, "XGrammar: Flexible and efficient structured generation for LLMs," arXiv:2411.15100, 2024.

[32] J. Muendler *et al.*, "Type-constrained code generation with language models," PLDI 2025, 2025.

[33] C. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" 2024.

[34] A. Dakhel *et al.*, "GitHub Copilot AI pair programmer: Asset or liability?" 2023.

[35] S. Lau and P. Guo, "From coding assistant to software developer: How novice programmers use LLMs," arXiv:2304.01984, 2023.

[36] J. Prather *et al.*, "It's complicated: Perspectives of novice programmers on GitHub Copilot," arXiv:2303.08733, 2023.

[37] M. Liffiton *et al.*, "CodeHelp: Using large language models with guardrails for programming classes," arXiv:2308.06921, 2023.

[38] E. Novak and B. McDanel, "Designing LLM-resistant programming assignments," SIGCSE TS 2025, 2025.

[39] M. Orenstrakh *et al.*, "Detecting LLM-generated code assignments using perplexity of code," arXiv:2307.09714, 2023.

[40] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," NeurIPS 2022 (arXiv:2203.02155), 2022.

[41] V. Krakovna *et al.*, "Specification gaming: The flip side of AI ingenuity," DeepMind Blog, 2020.